AVAGYM

TÉCNICO

AVAGYM · PWA DE ACADEMIA · VERSÃO V4.3 EM PRODUÇÃO · REVISÃO 2 (MAIN 63454EC)

Documentação Técnica

Arquitetura, infraestrutura, módulos, modelo de dados, API, permissões e escopo (R1–R13), segurança, service worker, testes, deploy, backup e configuração.

Sistema **AVAGYM V4.3 rev. 2 (base V4.2)**	Repositório **Trindadelucas0/acade · main**
Commit-base **63454ec**	Migrations **001–035**
Data **15/10/2026**	Sigilo **Sem segredos, tokens ou IPs**

Sumário

1. Visão geral técnica 2

2. Arquitetura 3

2.1 Componentes 3

2.2 Pipeline de uma requisição na API 3

2.3 Processos em segundo plano (na própria API) 4

3. Infraestrutura 5

4. Módulos 6

4.1 API (app/api) 6

4.2 Frontend (app/frontend/src) 7

5. Modelo de dados 8

5.1 Tabelas principais por domínio 8

5.2 Relações centrais 9

5.3 Migrations 9

6. Endpoints da API por área 11

7. Permissões por perfil 21

8. Fluxos técnicos críticos 22

8.1 Máquina de estados da sessão 22

8.2 Ficha nova substitui a anterior e um treino ativo (R10 + R13) 23

8.3 Decisão de acesso no servidor 23

8.4 LGPD — exportação por e-mail 24

9. Service worker e atualização 25

10. Segurança (rev. 2) 26

11. Testes 27

12. Deploy, backup e rollback 28

12.1 Atualização (roteiro) 28

12.2 Backup 28

12.3 Rollback 28

13. Configuração e variáveis de ambiente 29

13.1 Configuração pendente 29

13.2 Variáveis (somente nomes) 29

14. Itens a confirmar 30

1 Visão geral técnica

Este documento descreve a arquitetura e a operação do AVAGYM em produção (**academ.avadesk.com.br**). Fonte: repositório `Trindadelucas0/acade`, branch `main`, commit `63454ec` (revisão 2 deste documento). Conferido em 15/10/2026: a produção roda o mesmo commit, build `20261015233640`, e a API devolve os novos cabeçalhos de segurança.

Commit	O que entrou (cronológico)
9b84390	V4.2: contas do personal (senha provisória), cadeado + Solicitar liberação, ficha nova substitui anterior, LGPD por e-mail
50a01c1	V4.3: tours guiados por perfil (`user_tours` , migration 033)
8967a8a	Tema sol/lua no cabeçalho
3729817	**Novo nesta revisão**: regras de acesso R1–R12 (`scope.js` , migration 034), cadastro com código do personal, R6 (personal pendente), painel do usuário do admin, Liberação de contas unificada (abas + lote), tour do admin
de5cba9	**Novo**: R13 — um treino ativo por vez no servidor (`enforceOneActive` , migration 035)
3935401, 63454ec	**Novo**: segurança — cabeçalhos HTTP, limite de tentativas por IP, `HOST` configurável, nginx do docker com `server_tokens off`

> Este documento **não contém** senhas, tokens, chaves, IPs nem conteúdo do `.env`. As variáveis de ambiente aparecem só pelo nome.

Camada	Tecnologia	Onde no repositório
Frontend (PWA)	React 19 + Vite, react-router (HashRouter), zustand, three.js / @react-three (mapa 3D), Capacitor (builds Android/iOS opcionais)	`app/frontend`
API	Node.js com `http` nativo (sem framework), tabela de rotas `"MÉTODO /caminho"` , `pg` , `bcryptjs` , `jsonwebtoken` , `web-push`	`app/api`
Banco	PostgreSQL 16; migrations SQL numeradas 001–035 (034 = escopo da comunidade, 035 = um treino ativo) com scripts de down 003–035	`app/api/db/migrations` , `app/api/db/down`
Arquivos	Uploads de mídia e documentos profissionais privados no disco; `DATA_DIR` com segredo de sessão legado, chaves VAPID e `db.json` legado	`app/api/uploads` , `app/api/private` , `DATA_DIR`
Testes	API: scripts Node contra Postgres local; Frontend: vitest; E2E: Playwright (porta 4449, nunca produção)	`app/api/test` , `app/frontend/src/**/*.test.js` , `app/frontend/e2e`

Origem: fork do projeto openGym (licença MIT, ver `NOTICE.md`), estendido como AVADESK Fitness / AVAGYM.

2 Arquitetura

2.1 Componentes

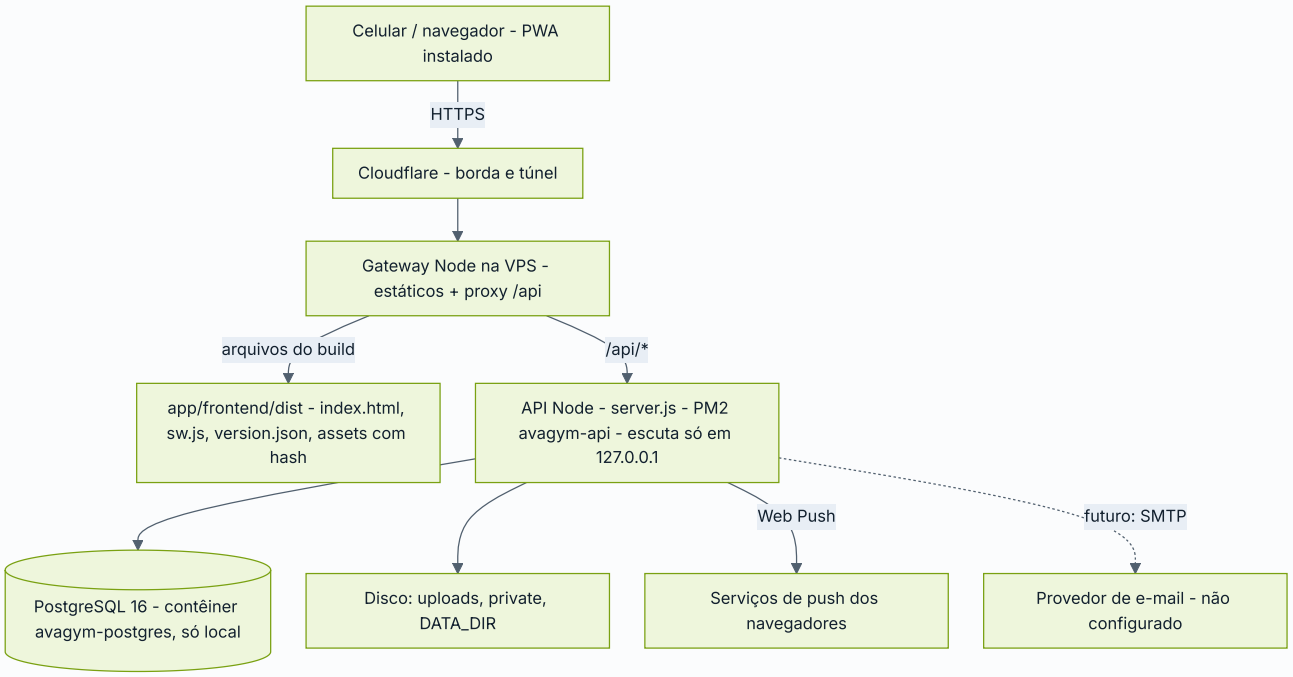


Figura 1 — visão de implantação (o gateway avagym-web e o túnel não estão no repositório).

2.2 Pipeline de uma requisição na API

1. `server.js` aplica os **cabeçalhos de segurança** em toda resposta e o **limite de tentativas** das rotas sensíveis (429 com `Retry-After`); depois monta a chave `MÉTODO + caminho` e procura o handler na tabela `routes` (404 se não existir).
2. `authUser` lê o JWT (cookie httpOnly `avagym_token` ou `Authorization: Bearer`).
3. `entitlementGate` (entitlements.js): regras por rota de plano/recurso e verificação profissional.
4. `accessGate` (accounts-v42.js): R6 primeiro (aluno coberto com personal pendente → toda escrita fora de `ALLOW_LOCKED` recebe 403 `ACCESS_LOCKED` com `reason: COACH_PENDING`), depois escopo do aluno do personal (`CLIENT_SCOPE`) e cadeado de liberação (`ACCESS_LOCKED`).
5. Escopo de conteúdo (scope.js), dentro dos handlers de comunidade e desafios: `visibleScopes` / `scopeCond` filtram feed e listas; `canSee` recusa acesso direto a item de outra comunidade (`CLIENT_SCOPE`); `newScope` decide para onde vai um post/desafio novo.
6. Handler: `requireAuth` / `requireRole` / `requireTrainee`, validação, SQL via `q()`, auditoria (`audit_log`) e notificações.
7. Erros de limite em transação viram respostas padronizadas (`LIMIT_REACHED` 409, `SUBSCRIPTION_INACTIVE` 402); demais exceções → 500 “server error”.

2.3 Processos em segundo plano (na própria API)

Processo	Intervalo	O que faz	Controle
Varredura de treino esquecido (`forgottenSweep`)	5 min	Sessões V4 abertas sem atividade ≥ N min → estado `aguardando_recuperacao` ; lembrete (notificação + push) a cada N min, com deduplicação	`SESSION_SWEEP` (desliga com o valor `off`), `SESSION_REMINDER_MIN` (padrão 60)
Fila de e-mail (`processOutbox`)	60 s	Envia `email_outbox` pendentes (até 5 tentativas) via nodemailer	Só roda com `SMTP_HOST` e `SMTP_FROM` ; pacote `nodemailer` ainda não instalado
Timers de descanso e lembrete diário	por usuário	Push “Rest over” quando o app está em segundo plano; lembrete de treino planejado	Inscrições de push em `db.json`

Divergência com a regra “tudo no Postgres”: as inscrições de Web Push (`db.subs`), a ponte de sessão legada (`db.users` , cookie `gymsid`) e as chaves VAPID/segredo continuam em arquivos no `DATA_DIR` ; a presença “treinando agora” fica em memória. A tabela `push_subscriptions` existe (001) mas não é usada pela API. A CONFIRMAR migrar ou aceitar.

3 Infraestrutura

Item	Situação (verificada em 15/10/2026)	Status
VPS	Servidor Linux próprio que hospeda gateway, API e banco. Acesso por SSH com chave (não documentado aqui).	Confirmado
Pasta da aplicação	`/opt/avagym`, clone da branch `main` (mesmo commit 63454ec). Manuais operacionais em `/opt/avagym/docs/`.	Confirmado (resolve a dúvida /opt/acade × /opt/avagym)
PM2	Processos `avagym-api` (API) e `avagym-web` (gateway/estáticos), salvos para subir sozinhos após reinício da VPS.	Confirmado
Gateway	Processo Node (fora do repositório) que serve o build e faz proxy de `/api`. `index.html`, `sw.js`, `version.json`, `manifest.json` com `no-cache, no-store`; `/assets/*` imutáveis. **Não aplica** os cabeçalhos de segurança da API ao HTML (ver seção Segurança).	Parcial — código não versionado
PostgreSQL	Contêiner Docker `avagym-postgres` (imagem postgres 16), publicado só no endereço local. Nome do banco via `DB_NAME`.	Confirmado; volume e backup automático a confirmar
API	Escuta só no endereço local (`HOST=127.0.0.1`); só o gateway/túnel chega nela.	Confirmado
Cloudflare	Domínio atrás da Cloudflare; acesso de origem por túnel Cloudflare criado pelo Lucas. O IP real do cliente chega em `cf-connecting-ip` (usado pelo limite de tentativas).	A CONFIRMAR nome do túnel e regras
Domínio	`academ.avadesk.com.br` (definitivo); `ORIGIN` e `RP_ID` apontam para ele.	Confirmado
Docker Compose do repositório	`app/docker-compose.yml` + nginx do repositório (agora com `server_tokens off` e cabeçalhos). Não é o que roda em produção.	Informativo

4 Módulos

4.1 API (app/api)

Arquivo	Responsabilidade
server.js	Servidor HTTP, tabela de rotas, gates, push (VAPID), presença, timers, ponte legada
auth.js	Cadastro (com `pro_code` opcional, validado antes de criar a conta), login (bcrypt 12 + JWT, TTL `JWT_TTL_HOURS` padrão 168 h), troca de senha, sair de todos (`token_version`), recuperação via link do admin
accounts-v42.js	Aluno do personal (senha provisória), `attachByProCode` (cadastro com código), cadeado `accessGate` + `coachPending` (R6), pedidos de liberação, admin de acesso (`GET /api/adm/access` devolve também `unlocked_users` e `was_blocked` ; liberar/travar em lote), exportação LGPD por e-mail, ponto de extensão `PAYMENT`
scope.js	**Novo (3729817)**: escopo sistema × personal — `visibleScopes(u)` , `scopeCond` , `canSee(u, scope_pro_id)` , `newScope(u, pedido)` . `NULL` = sistema; preenchido = comunidade daquele personal
active-training.js	Treino ativo único; **R13 (de5cba9)**: `enforceOneActive` (ficha do personal tem prioridade, programa do sistema vira `switched`), `STUDENT_CAN_DETACH_PRO=true` condicionado à mensalidade própria (`can_detach`); coberto → `PRO_DETACH_ONLY` ; tema (`theme_pref`)
guided.js	Questionário, equipamentos, recomendações, catálogo, iniciar/abandonar programa, hoje, progresso, carga muscular
sessions-v4.js + session-machine.js	Início de sessão autorizado pelo treino ativo (`ONE_ACTIVE` , R13), eventos idempotentes, sessão aberta, correção de tempo, varredura de treino esquecido
programs-editorial.js	Programas públicos: rascunho, envio, revisão, publicação, selos, revogação
pro.js / portal.js	Fichas (pro_plans) e substituição, atendimento (ocorrências, conversas, propostas), notificações, painel PRO, prescrições, capacidade, solicitações, KPIs admin
entitlements.js / saas.js / invites.js	Motor de planos, assinaturas, concessões, gate de recursos; convites; código do aluno
identity.js	Contexto (treinar × atender), capacidades, verificação profissional e documentos
community.js, groups.js, challenges*.js, prizes.js, ranking.js	Feed e moderação, grupos, desafios (regras, feed, comprovantes, pontuação), troféus/prêmios, ranking
lgpd.js	Termos versionados, aceite, pedidos de exclusão, montagem do arquivo de exportação (`buildExport`)
tours.js	V4.3: estado dos tours guiados por usuário (`GET/POST /api/me/tours` , `POST /api/me/tours/reset`); chave `perfil:tela` ; status concluido/pulado/reiniciado
mailer.js	Fila `email_outbox` e envio SMTP (quando configurado)
appstate.js, legacy-import.js	Estado do app no Postgres (`user_app_state`); R13: `PUT /api/data` põe treino livre legado em quarentena (`user_app_state_rejections` , `free_workout_archived`); sessão ao vivo, importação do legado
admcrud.js, exmedia.js	Gestão administrativa (editar/arquivar), mídia de exercícios
db/	`migrate.js` (cria banco, aplica migrations em transação, seed opcional), `pool.js` , seeds, `backup.ps1`
scripts/	`create-admin` , `importar-legado` , `importar-estado-legado` , `remover-usuarios` (contas de teste), `zerar-dados`

4.2 Frontend (app/frontend/src)

Pasta / arquivo	Conteúdo
App.jsx	Shell, portões globais (troca de senha → termos → app), três árvores de rotas: admin, profissional, aluno; `LockBanner`, `ScopeGate`
views/guided	Today (Início), Train (motor único), Done, Progress, Profile, Programs, MyProgram, Onboarding, MyEquipment, SessionRecovery, AdminPrograms, TabBar
views/app	Treinos, Comunidade, Desafios, Grupos, Ranking, Avisos (Notificações), Acompanhamento, Trofeus, Terms, Locked, AdminPanel
views/pro, views/builder	Portal PRO, programas públicos, construtor de ficha
views/admin	AdminApp (menu em seções), AdminCrud e **UserPanel.jsx** (novo: Resumo, Mensalidade, Assinatura do personal, Treino atual e vínculos, Ações, Detalhes avançados)
views/v3/V42.jsx	MustChangePassword, UnlockRequest, LockBanner (textos de R2/R6/R9), AccessLockHost, ScopeGate, ProClients, PlanHistory, **AdmLiberacoes** (abas Pedidos/Aguardando/Liberadas/Travadas, seleção e lote; sai o formulário “Travar uma conta”), exportação LGPD
components/Tour.jsx, lib/tours.js	V4.3: `TourHost` decide quando abrir (1ª vez na tela, depois das boas-vindas; nunca em /treinar, /concluido, /pro/treino, /onboarding nem com sessão local aberta; desligado sob automação, salvo `tours_e2e=1`); spotlight acessível (Esc, setas, foco preso); passos por perfil; **admin**: boas-vindas de 6 passos + telas usuarios, usuario (8 passos), liberacoes, verificacoes, assinaturas, lgpd; no celular abre o menu (`reveal adm-menu`)
components/ActiveTraining.jsx	Cartão do treino ativo: “Programa pausado: …” e “Desvincular ficha do personal” (com confirmação)
components/ThemeToggle.jsx	V4.3: botão sol/lua no cabeçalho; alterna claro ↔ escuro e salva em `users.theme_pref`; `localStorage avagym_theme_cache` é só cache visual (tela de login)
lib/	api.js, guided.js (sessão local, fila de eventos), session-machine.js (cópia da do servidor), pwa-update.js, pwa-rules.js, theme.js, outbox.js, timing.js, wakelock.js, push.js, features.js
views/*.jsx (raiz)	Telas herdadas do openGym (Workout, History, Stats, Library, Settings). Treino livre desligado (`SHOW_FREESTYLE=false`)
public/sw.js, manifest.json	Service worker e manifesto do PWA

5 Modelo de dados

5.1 Tabelas principais por domínio

Domínio	Tabelas
Identidade e acesso	`tenants`, `users` (account_type `normal|personal_client`, must_change_password, managed_by, access_unlocked_at/by, theme_pref, token_version, status), `roles`, `user_roles`, `tenant_memberships`, `user_tours` (V4.3), `password_resets`, `unlock_requests`, `access_events`, `audit_log`
Profissional	`professional_profiles`, `professional_verifications`, `professional_documents`, `professional_registry_claims`, `professional_qualifications`, `coach_students` (vínculo), `pro_invites`, `professional_capacity_reconciliations`
Planos SaaS	`plans`, `plan_versions`, `subscriptions`, `subscription_events`, `entitlement_grants`, `usage_counters`, `access_plans`, `user_access_plans`
Catálogo	`exercises`, `exercise_muscles`, `muscles`, `muscle_groups`, `equipment`, `exercise_media`, `programs`, `program_workouts`, `program_workout_exercises`, `program_revisions`, `program_reviews`, `program_badges`, `program_equipment`
Treino ativo e fichas	`user_programs`, `user_onboarding`, `user_equipment`, `pro_plans` (status, superseded_by, archived_at), `pro_plan_history`, `workout_templates`, `student_active_training`, `student_active_training_history`, `active_training_fixes` (log da 035), `user_app_state_rejections`
Sessões	`sessions` (state, origin, server_started_at, last_activity_at, time_corrected_at, voided_at…), `session_sets`, `session_events`, `session_time_corrections`, `session_reminders`, `session_pain_reports`, `program_session_versions`
Atendimento	`occurrences`, `occurrence_assessments`, `threads`, `messages`, `change_proposals`, `pro_requests`, `notifications` (dedupe_key), `notification_prefs`
Comunidade e desafios	`posts` (+ `scope_pro_id`), `challenges` (+ `scope_pro_id`), `post_likes`, `post_comments`, `content_reports`, `follows`, `groups`, `group_members`, `challenges` + regras/equipes/membros/feed/comprovantes/resultados, `ranking_*`, `trophy_templates`, `user_trophies`, `prizes`
LGPD e e-mail	`terms_versions`, `term_acceptances`, `data_requests` (exclusão), `data_export_requests`, `email_outbox`
Estado e legado	`user_app_state`, `guided_live_sessions`, `legacy_routines`, `legacy_custom_exercises`, `legacy_session_sets`, `legacy_imports`

5.2 Relações centrais

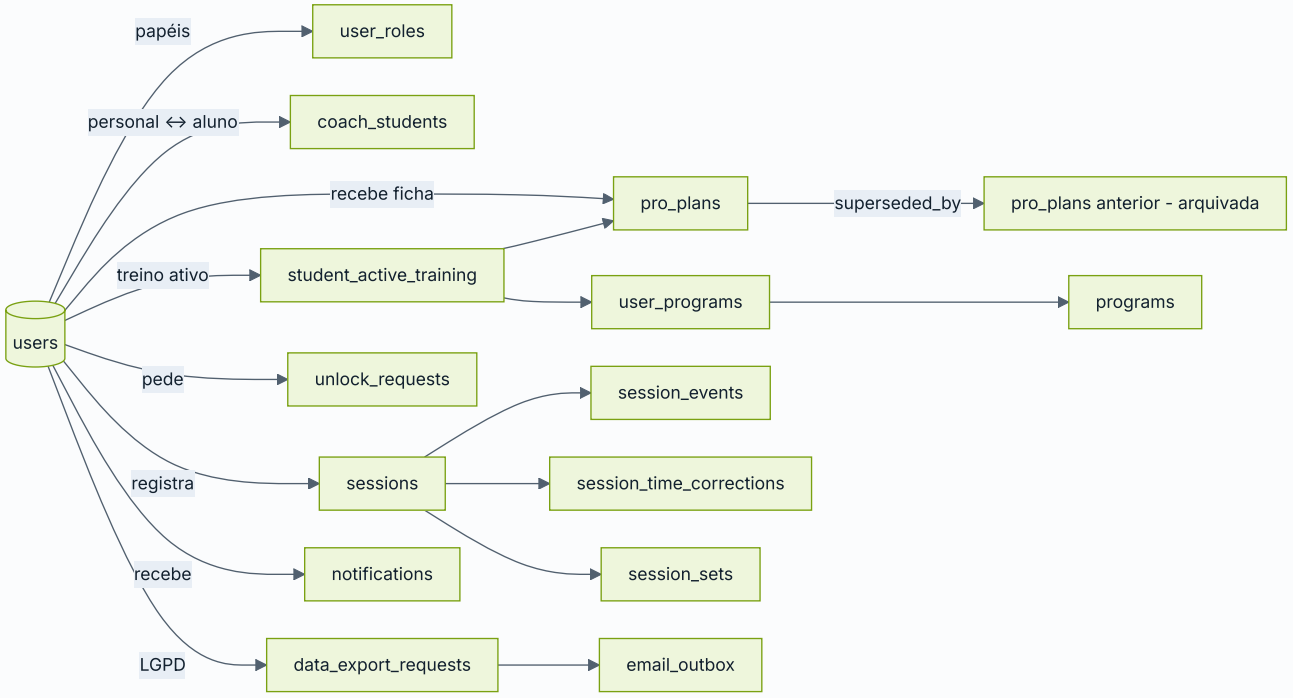


Figura 2 — relações principais (simplificado).

5.3 Migrations

Nº	Arquivo	Conteúdo
001	`001_init.sql`	Base: tenants, usuários, papéis, catálogo, programas, sessões, push
002	`002_guided_training.sql`	Questionário e treino guiado
003	`003_contas_auditoria.sql`	Planos de acesso, auditoria
004	`004_profissionais_atendimento.sql`	Perfis profissionais, fichas, atendimento, notificações
005	`005_comunidade_ranking.sql`	Feed, denúncias, ranking
006	`006_recuperacao_senha.sql`	Links de redefinição
007	`007_termos_lgpd.sql`	Termos e pedidos LGPD
008	`008_midia_exercicios_tenant.sql`	Mídia de exercícios
009	`009_portal_profissional.sql`	Modelos, solicitações PRO
010	`010_admin_gestao.sql`	Anulação de sessões, arquivamento
011	`011_construtor_treino.sql`	Construtor de fichas
012	`012_tempo_treino.sql`	Tempos de série/descanso
013	`013_grupos.sql`	Grupos e convites
014	`014_desafios.sql`	Desafios
015	`015_desafios_feed_comprovantes.sql`	Feed e comprovantes
016	`016_desafios_ranking.sql`	Resultados
017	`017_trofeus_premiacoes.sql`	Troféus e prêmios
018	`018_identidade_global.sql`	Identidade única, verificação
019	`019_motor_planos.sql`	Planos SaaS, concessões

Nº	Arquivo	Conteúdo
020	`020_planos_gratuitos.sql`	Convites do Profissional Free
021	`021_saas_aberto_codigo_aluno.sql`	Cadastro aberto, código do aluno
022	`022_fim_pedido_vinculo_admin.sql`	Fim do pedido de vínculo via admin
023	`023_estado_app_postgres.sql`	Estado do app no Postgres
024	`024_unicidade.sql`	Unicidade de registros
025	`025_sessao_estados_eventos.sql`	Estados/eventos da sessão, correção de tempo, lembretes
026	`026_legado_importado.sql`	Histórico legado
027	`027_programas_editorial.sql`	Revisões, avaliações, selos
028	`028_equipamentos.sql`	Equipamentos do usuário/programa
029	`029_capacidade_reconciliacao.sql`	Reconciliação de vagas
030	`030_notificacoes_sessoes.sql`	dedupe_key, token_version
031	`031_tema_treino_ativo.sql`	theme_pref, treino ativo único
032	`032_aluno_personal_lgpd_email.sql`	V4.2: account_type, senha provisória, liberação, superseded_by, email_outbox, data_export_requests, access_events; libera contas existentes
033	`033_user_tours.sql`	V4.3: progresso dos tours guiados por usuário
034	`034_escopo_comunidade.sql`	**Rev. 2**: `posts.scope_pro_id` e `challenges.scope_pro_id` (NULL = sistema; tudo o que existia ficou do sistema)
035	`035_um_treino_ativo.sql`	**Rev. 2 — atual em produção**: normaliza dados antigos para um treino ativo, sem apagar (programa vira `switched`; log em `active_training_fixes`; down restaura)

Regras: migrations aplicadas nunca são alteradas; toda mudança é nova migration aditiva com down em `db/down/0NN_down.sql` (existem de 003 a 035). `migrate.js` grava em `schema_migrations` e aplica cada arquivo em transação.

Achado na homologação desta revisão: rodar `migrate.js` num banco **vazio** falha na 028_equipamentos, porque ela depende de um item do seed (equipamento “elastico”) que só é inserido depois de todas as migrations. Em produção não afeta (banco já existe). Contorno usado: aplicar até a 027, rodar o seed e depois o restante. A CONFIRMAR corrigir a ordem no migrate.js ou na 028

6 Endpoints da API por área

Total de **311** rotas (V4.2 + V4.3). A revisão 2 **não criou rotas novas**: mudaram regras e respostas (cadastro com `pro_code`, escopo nos handlers de comunidade/desafios, `GET /api/adm/access` com mais listas, R13 em treino ativo e sessões). "Acesso" é uma indicação pelo prefixo; a regra final é a do handler e dos gates (seção 7). Rotas GET de leitura são liberadas para contas travadas; para o aluno do personal, catálogo/recomendações, ranking e grupos são bloqueados.

Autenticação e conta `auth.js · 13`

Método	Caminho	Acesso
POST	`/api/auth/register`	autenticado
POST	`/api/auth/login`	autenticado
POST	`/api/auth/logout`	autenticado
POST	`/api/auth/logout-all`	autenticado
POST	`/api/auth/password`	autenticado
GET	`/api/auth/me`	autenticado
POST	`/api/auth/forgot`	autenticado
POST	`/api/auth/reset`	autenticado
POST	`/api/adm/users/reset-link`	admin
GET	`/api/muscles`	público/autenticado
GET	`/api/muscles/exercises`	público/autenticado
GET	`/api/programs`	autenticado
GET	`/api/admin/tenant-users`	admin

Contas V4.2 (aluno do personal, liberação, LGPD por e-mail) `accounts-v42.js · 10`

Método	Caminho	Acesso
POST	`/api/pro/clients`	profissional
GET	`/api/pro/clients`	profissional
POST	`/api/pro/clients/reset-password`	profissional
GET	`/api/me/unlock-request`	autenticado
POST	`/api/me/unlock-request`	autenticado
GET	`/api/adm/access`	admin
POST	`/api/adm/access/decide`	admin
POST	`/api/adm/access/set`	admin
GET	`/api/me/export-request`	autenticado
POST	`/api/me/export-request`	autenticado

Identidade, contexto e verificação profissional `identity.js · 9`

Método	Caminho	Acesso
GET	`/api/me/context`	autenticado
POST	`/api/me/context`	autenticado
POST	`/api/me/capacities`	autenticado

Método	Caminho	Acesso
GET	`/api/pro/verification`	profissional
POST	`/api/pro/verification`	profissional
POST	`/api/pro/verification/document`	profissional
GET	`/api/verification/document`	autenticado
GET	`/api/adm/verifications`	admin
POST	`/api/adm/verifications/decide`	admin

Treino ativo e tema `active-training.js · 5`

Método	Caminho	Acesso
GET	`/api/active-training`	autenticado
POST	`/api/active-training`	autenticado
POST	`/api/active-training/detach`	autenticado
GET	`/api/me/theme`	autenticado
POST	`/api/me/theme`	autenticado

Treino guiado (questionário, programas, progresso) `guided.js · 15`

Método	Caminho	Acesso
GET	`/api/guided/state`	autenticado
POST	`/api/guided/onboarding`	autenticado
GET	`/api/guided/equipment`	autenticado
PUT	`/api/guided/equipment`	autenticado
GET	`/api/guided/recommendations`	autenticado
GET	`/api/guided/programs`	autenticado
GET	`/api/guided/program`	autenticado
POST	`/api/guided/start`	autenticado
POST	`/api/guided/abandon`	autenticado
GET	`/api/guided/today`	autenticado
POST	`/api/guided/sessions`	autenticado
GET	`/api/guided/progress`	autenticado
GET	`/api/guided/muscle-load`	autenticado
GET	`/api/guided/admin/programs`	autenticado
POST	`/api/guided/admin/program-status`	autenticado

Sessões V4 (máquina de estados) `sessions-v4.js · 5`

Método	Caminho	Acesso
POST	`/api/guided/sessions/start`	autenticado
POST	`/api/guided/sessions/events`	autenticado
GET	`/api/guided/sessions/open`	autenticado
GET	`/api/guided/sessions/events`	autenticado
POST	`/api/guided/sessions/correct-time`	autenticado

Programas — fluxo editorial programs-editorial.js · 13

Método	Caminho	Acesso
GET	`/api/pro/programs`	profissional
GET	`/api/pro/program`	profissional
POST	`/api/pro/programs`	profissional
PATCH	`/api/pro/programs`	profissional
POST	`/api/pro/programs/submit`	profissional
GET	`/api/admin/program-reviews`	admin
GET	`/api/admin/program-review`	admin
POST	`/api/admin/program-reviews/decision`	admin
POST	`/api/admin/programs/publish`	admin
POST	`/api/admin/programs/archive`	admin
POST	`/api/admin/programs/revoke`	admin
POST	`/api/admin/programs/badges`	admin
POST	`/api/admin/program-badges/revoke`	admin

Profissional, atendimento, notificações e admin base pro.js · 45

Método	Caminho	Acesso
GET	`/api/care`	autenticado
GET	`/api/plan`	autenticado
GET	`/api/blocks`	autenticado
POST	`/api/occurrences`	autenticado
POST	`/api/threads`	autenticado
GET	`/api/thread`	autenticado
POST	`/api/messages`	autenticado
GET	`/api/notifications`	autenticado
POST	`/api/notifications/read`	autenticado
GET	`/api/professionals`	autenticado
GET	`/api/professional`	autenticado
GET	`/api/exercises/search`	autenticado
GET	`/api/pro/dashboard`	profissional
POST	`/api/pro/profile`	profissional
POST	`/api/pro/qualifications`	profissional
GET	`/api/pro/student`	profissional
POST	`/api/pro/plans`	profissional
POST	`/api/pro/plans/archive`	profissional
POST	`/api/pro/assess`	profissional
POST	`/api/pro/close-occurrence`	profissional
POST	`/api/pro/close-thread`	profissional
POST	`/api/pro/proposals`	profissional
POST	`/api/pro/proposals/decide`	profissional

Método	Caminho	Acesso
GET	`/api/adm/overview`	admin
GET	`/api/adm/users`	admin
POST	`/api/adm/users/status`	admin
POST	`/api/adm/users/role`	admin
POST	`/api/adm/users/create`	admin
POST	`/api/adm/users/same-person`	admin
POST	`/api/adm/users/plan`	admin
GET	`/api/adm/plans`	admin
POST	`/api/adm/plans`	admin
GET	`/api/adm/professionals`	admin
POST	`/api/adm/professionals`	admin
POST	`/api/adm/qualifications/verify`	admin
GET	`/api/adm/links`	admin
POST	`/api/adm/links`	admin
POST	`/api/adm/links/end`	admin
GET	`/api/adm/queue`	admin
POST	`/api/adm/assign`	admin
POST	`/api/adm/close-occurrence`	admin
GET	`/api/adm/audit`	admin
GET	`/api/adm/settings`	admin
POST	`/api/adm/settings`	admin
POST	`/api/pro/threads`	profissional

Portal PRO, prescrições, capacidade e solicitações portal.js · 23

Método	Caminho	Acesso
GET	`/api/pro/home`	profissional
GET	`/api/pro/students`	profissional
GET	`/api/pro/student/full`	profissional
GET	`/api/pro/library`	profissional
GET	`/api/pro/templates`	profissional
POST	`/api/pro/templates`	profissional
GET	`/api/pro/prescriptions`	profissional
GET	`/api/pro/prescription`	profissional
POST	`/api/pro/prescriptions`	profissional
POST	`/api/pro/prescriptions/publish`	profissional
POST	`/api/pro/prescriptions/archive`	profissional
GET	`/api/pro/capacity`	profissional
POST	`/api/pro/requests`	profissional
POST	`/api/pro/requests/cancel`	profissional
POST	`/api/pro/requests/reply`	profissional

Método	Caminho	Acesso
GET	`/api/me/code`	autenticado
GET	`/api/adm/requests`	admin
POST	`/api/adm/requests/decide`	admin
POST	`/api/adm/capacity`	admin
GET	`/api/adm/templates`	admin
POST	`/api/adm/templates`	admin
POST	`/api/adm/templates/status`	admin
GET	`/api/adm/kpis`	admin

Planos, assinaturas e concessões entitlements.js · 13

Método	Caminho	Acesso
POST	`/api/pro/prescriptions/publish`	profissional
POST	`/api/pro/prescriptions`	profissional
POST	`/api/pro/plans`	profissional
POST	`/api/pro/invites`	profissional
POST	`/api/invites/accept`	autenticado
GET	`/api/plans`	público/autenticado
GET	`/api/me/subscriptions`	autenticado
GET	`/api/adm/subscriptions`	admin
POST	`/api/adm/subscriptions/plan`	admin
POST	`/api/adm/subscriptions/status`	admin
POST	`/api/adm/grants`	admin
POST	`/api/adm/grants/revoke`	admin
GET	`/api/adm/grants`	admin

Convites invites.js · 7

Método	Caminho	Acesso
GET	`/api/pro/invites`	profissional
POST	`/api/pro/invites`	profissional
POST	`/api/pro/invites/cancel`	profissional
GET	`/api/me/invites`	autenticado
GET	`/api/invites/view`	público/autenticado
POST	`/api/invites/accept`	autenticado
POST	`/api/invites/decline`	autenticado

SaaS (código do aluno, visão 360°) saas.js · 2

Método	Caminho	Acesso
GET	`/api/pro/students/lookup`	profissional
GET	`/api/adm/users/overview`	admin

Comunidade (feed) community.js · 14

Método	Caminho	Acesso
GET	`/api/feed/categories`	público/autenticado
GET	`/api/feed`	autenticado
GET	`/api/post`	autenticado
POST	`/api/posts`	autenticado
POST	`/api/posts/delete`	autenticado
POST	`/api/posts/like`	autenticado
POST	`/api/comments`	autenticado
POST	`/api/comments/delete`	autenticado
POST	`/api/reports`	autenticado
POST	`/api/follows`	autenticado
POST	`/api/uploads`	autenticado
GET	`/api/media`	autenticado
GET	`/api/adm/reports`	admin
POST	`/api/adm/reports/decide`	admin

Grupos groups.js · 14

Método	Caminho	Acesso
GET	`/api/groups`	autenticado
GET	`/api/group`	autenticado
POST	`/api/groups`	autenticado
POST	`/api/groups/update`	autenticado
POST	`/api/groups/join`	autenticado
POST	`/api/groups/leave`	autenticado
POST	`/api/groups/decide`	autenticado
POST	`/api/groups/remove`	autenticado
POST	`/api/groups/role`	autenticado
POST	`/api/groups/transfer`	autenticado
POST	`/api/groups/invites`	autenticado
POST	`/api/groups/invites/revoke`	autenticado
GET	`/api/adm/groups`	admin
POST	`/api/adm/groups/status`	admin

Desafios challenges.js · 16

Método	Caminho	Acesso
GET	`/api/challenge-categories`	público/autenticado
POST	`/api/adm/challenge-categories`	admin
GET	`/api/challenges`	autenticado
GET	`/api/challenge`	autenticado
POST	`/api/challenges`	autenticado

Método	Caminho	Acesso
POST	`/api/challenges/publish`	autenticado
POST	`/api/challenges/transition`	autenticado
POST	`/api/challenges/join`	autenticado
POST	`/api/challenges/leave`	autenticado
POST	`/api/challenges/decide`	autenticado
POST	`/api/challenges/remove`	autenticado
POST	`/api/challenges/invite-students`	autenticado
POST	`/api/challenges/invites`	autenticado
POST	`/api/challenges/invites/revoke`	autenticado
POST	`/api/challenges/optout`	autenticado
GET	`/api/adm/challenges`	admin

Desafios — feed, comprovantes e ranking challenge-feed.js · 23

Método	Caminho	Acesso
GET	`/api/notification-prefs`	autenticado
POST	`/api/notification-prefs`	autenticado
POST	`/api/challenges/media`	autenticado
POST	`/api/challenges/media/stream`	autenticado
GET	`/api/challenges/feed`	autenticado
POST	`/api/challenges/posts`	autenticado
POST	`/api/challenges/posts/publish`	autenticado
POST	`/api/challenges/posts/delete`	autenticado
POST	`/api/challenges/posts/pin`	autenticado
POST	`/api/challenges/posts/like`	autenticado
POST	`/api/challenges/posts/comment`	autenticado
GET	`/api/challenges/ranking`	autenticado
GET	`/api/challenges/score`	autenticado
GET	`/api/challenges/sessions`	autenticado
POST	`/api/challenges/proofs`	autenticado
GET	`/api/challenges/proofs`	autenticado
POST	`/api/challenges/proofs/review`	autenticado
POST	`/api/challenges/proofs/replace`	autenticado
POST	`/api/challenges/proofs/appeal`	autenticado
POST	`/api/adm/proofs/invalidate`	admin
GET	`/api/adm/proofs`	admin
GET	`/api/adm/proof-reviewers`	admin
POST	`/api/adm/proof-reviewers`	admin

Troféus e premiações prizes.js · 16

Método	Caminho	Acesso
GET	/api/trophies	autenticado
POST	/api/trophies	autenticado
GET	/api/adm/trophies	admin
POST	/api/adm/trophies/decide	admin
GET	/api/challenges/trophies	autenticado
POST	/api/challenges/trophies	autenticado
GET	/api/me/trophies	autenticado
POST	/api/me/trophies/show	autenticado
GET	/api/users/trophies	autenticado
GET	/api/challenges/prizes	autenticado
GET	/api/adm/prizes	admin
POST	/api/adm/prizes	admin
POST	/api/adm/prizes/transition	admin
POST	/api/adm/challenges/recompute	admin
POST	/api/adm/challenges/homologate	admin
POST	/api/adm/prizes/deliver	admin

Ranking ranking.js · 5

Método	Caminho	Acesso
GET	/api/ranking	autenticado
GET	/api/ranking/history	autenticado
POST	/api/ranking/prefs	autenticado
GET	/api/adm/ranking	admin
POST	/api/adm/ranking/invalidate	admin

Termos e LGPD lgpd.js · 9

Método	Caminho	Acesso
GET	/api/terms	público/autenticado
POST	/api/terms/accept	autenticado
POST	/api/adm/terms	admin
GET	/api/adm/terms	admin
GET	/api/me/requests	autenticado
POST	/api/me/delete-request	autenticado
POST	/api/me/delete-request/cancel	autenticado
GET	/api/adm/data-requests	admin
POST	/api/adm/data-requests/decide	admin

Mídia de exercícios exmedia.js · 5

Método	Caminho	Acesso
GET	/api/exercises/media	autenticado
GET	/api/adm/exercises	admin
POST	/api/adm/exercises/media	admin
POST	/api/adm/exercises/upload	admin
POST	/api/adm/exercises/media/remove	admin

Admin — gestão (CRUD) admcrud.js · 25

Método	Caminho	Acesso
POST	/api/adm/users/update	admin
POST	/api/adm/users/delete	admin
POST	/api/adm/professionals/profile	admin
GET	/api/adm/qualifications	admin
POST	/api/adm/qualifications/save	admin
POST	/api/adm/qualifications/delete	admin
GET	/api/adm/student/record	admin
POST	/api/adm/student/record	admin
POST	/api/adm/restrictions/save	admin
POST	/api/adm/restrictions/delete	admin
GET	/api/adm/student/training	admin
POST	/api/adm/plans/save	admin
GET	/api/adm/plan	admin
POST	/api/adm/plans/delete	admin
POST	/api/adm/user-programs/end	admin
POST	/api/adm/sessions/void	admin
POST	/api/adm/links/transfer	admin
GET	/api/adm/posts	admin
POST	/api/adm/posts/update	admin
POST	/api/adm/posts/delete	admin
GET	/api/adm/comments	admin
POST	/api/adm/comments/update	admin
POST	/api/adm/comments/delete	admin
POST	/api/adm/exercises/save	admin
POST	/api/adm/exercises/archive	admin

Estado do app e legado appstate.js · 7

Método	Caminho	Acesso
GET	/api/data	autenticado
PUT	/api/data	autenticado
GET	/api/me/legacy	autenticado

Método	Caminho	Acesso
GET	`/api/me/live-session`	autenticado
PUT	`/api/me/live-session`	autenticado
POST	`/api/me/live-session/clear`	autenticado
POST	`/api/me/onboarding/skip`	autenticado

Tours guiados (V4.3) tours.js · 3

Método	Caminho	Acesso
GET	`/api/me/tours`	autenticado
POST	`/api/me/tours`	autenticado
POST	`/api/me/tours/reset`	autenticado

Infra do servidor (saúde, push, presença) server.js · 14

Método	Caminho	Acesso
GET	`/api/health`	público/autenticado
GET	`/api/config`	público/autenticado
GET	`/api/me`	autenticado
POST	`/api/logout`	autenticado
POST	`/api/logout/all`	autenticado
GET	`/api/push/public-key`	público/autenticado
POST	`/api/push/subscribe`	autenticado
POST	`/api/push/unsubscribe`	autenticado
POST	`/api/push/test`	autenticado
POST	`/api/push/rest-timer`	autenticado
POST	`/api/push/rest-timer/cancel`	autenticado
POST	`/api/activity`	autenticado
GET	`/api/adm/live`	admin
GET	`/api/adm/users/workouts`	admin

> Removido na V4.1: `GET /api/me/export` (download direto) — responde 404. A cópia LGPD só sai por e-mail (`POST /api/me/export-request`).

7 Permissões por perfil

Mecanismo	Regra
`requireAuth`	Exige JWT válido; bloqueia `must_change_password` (exceto rotas com `allowPending`) e status ≠ ativo.
`requireRole(r)`	403 `PERMISSION_DENIED` sem o papel (aluno, profissional, admin).
`requireTrainee`	Treino só para quem tem papel aluno (admin ou profissional puros são recusados).
`entitlementGate`	Rotas com regra de plano: exige verificação profissional e recurso do plano (`FEATURE_NOT_INCLUDED`, `LIMIT_REACHED`, `SUBSCRIPTION_INACTIVE`). Admin passa direto.
`accessGate` — R6	Aluno coberto (`personal_client` sem liberação própria) cujo personal está pendente (`coach_unlocked_at` nulo): escritas fora de `ALLOW_LOCKED` → 403 `ACCESS_LOCKED`, `reason: COACH_PENDING`. O app recebe `coach_pending` no perfil e mostra o aviso.
`accessGate` — conta travada	Se `access_unlocked_at` é nulo (e não é admin): todo GET passa; escritas só na lista `ALLOW_LOCKED` (auth, termos, tema, contexto, pedido de liberação, LGPD, preferências, questionário, push, logout, verificação profissional, tours) → senão 403 `ACCESS_LOCKED`.
`accessGate` — aluno do personal	`account_type=personal_client` e não liberado: GET de catálogo/recomendações, ranking e grupos → 403 `CLIENT_SCOPE`; escritas permitidas em `CLIENT_SCOPE`: sessões, treino ativo, ocorrências/conversas/mensagens, sessão ao vivo, curtir/comentar, desafios (entrar, sair, comprovantes, posts).
Escopo de conteúdo (scope.js)	Usuário normal liberado: sistema + personais vinculados. Aluno coberto: só o personal dele. Aluno com mensalidade própria: os dois. Personal: o dele + sistema. Admin: tudo. Item fora do escopo → 403 `CLIENT_SCOPE`.
Treino ativo (R13)	`guided/start` 409 `MUST_DETACH` (pode desvincular) ou `COACH_LOCKED` (coberto); desvincular ficha → `PRO_DETACH_ONLY` sem mensalidade própria (`STUDENT_CAN_DETACH_PRO=true`); sessão fora do treino ativo → `ONE_ACTIVE`; `SESSION_OPEN`.
Cadastro com código	`attachByProCode`: personal ativo, com papel profissional, **verificado** e com vaga (`capacitySummary`); senão recusa antes de criar a conta. Cria `coach_students` com `origin=codigo` e notifica o personal.
Admin	Não trava a si mesmo; não aprova o próprio programa; decisões auditadas.

Capacidade	Aluno do personal	Usuário normal	Personal	Admin
Treinar	ficha ativa	após liberação	com papel aluno	com papel aluno
Catálogo do sistema	403 (sim com mensalidade própria)	sim	sim	sim
Comunidade/desafios	só do personal	sistema + vinculados	o dele + sistema	tudo
Cadastrar aluno `POST /api/pro/clients`	—	—	verificado + liberado + vaga	sim
Fichas `/api/pro/plans`	—	—	verificado + plano	sim
Liberação `/api/adm/access*`	—	—	—	sim
LGPD (cópia/exclusão)	sim	sim	sim	cópia sim; exclusão não

8 Fluxos técnicos críticos

8.1 Máquina de estados da sessão

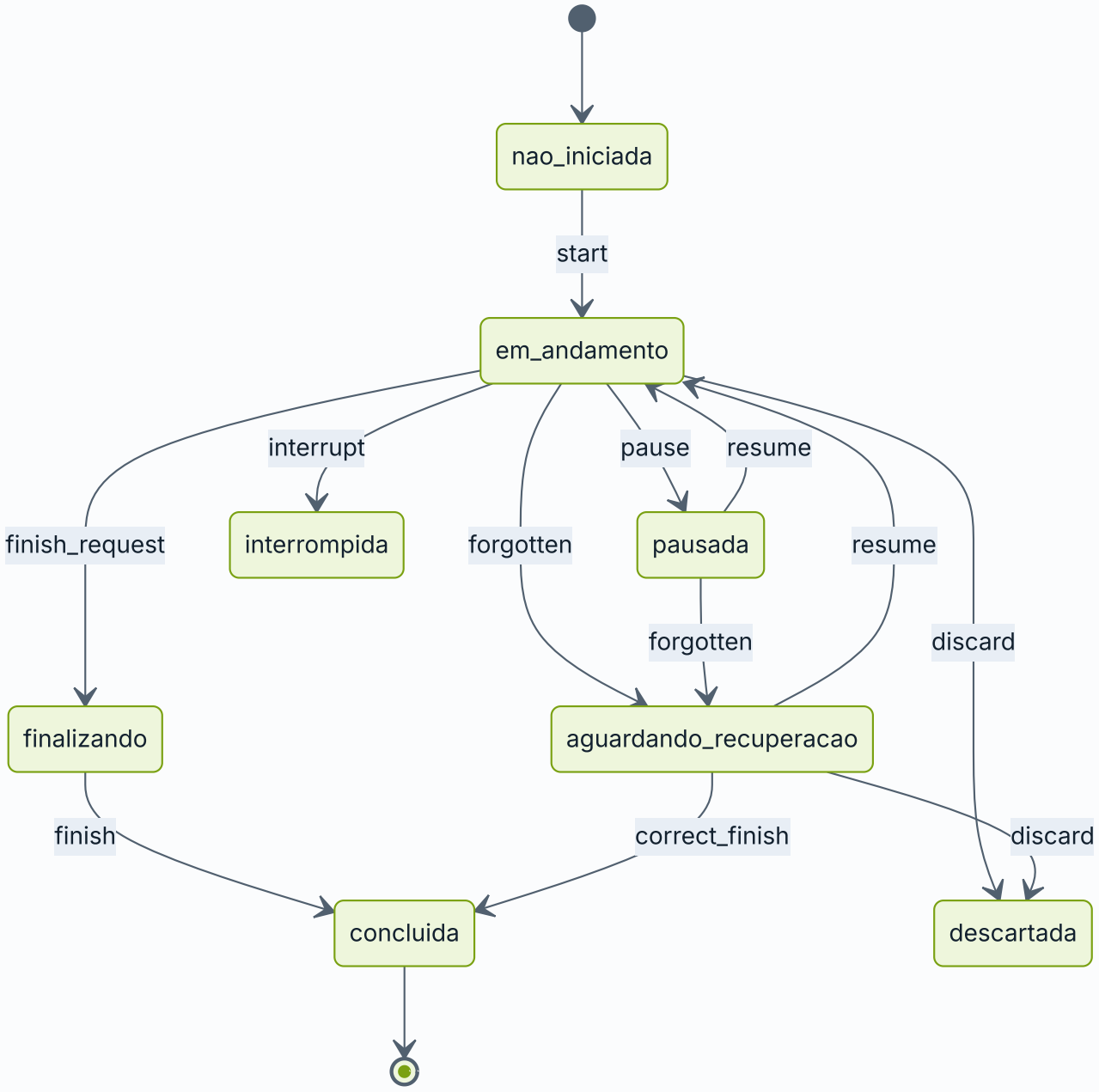


Figura 3 — estados (session-machine.js, idêntico no app e na API). Eventos de atividade: set_start, set_done, set_edit, rest_*, exercise_change, heartbeat.

- Início exige o treino ativo; uma sessão aberta por vez (`SESSION_OPEN` , salvo `force_new`).
- Eventos idempotentes por `client_event_id` ; o app guarda numa fila local (`outbox`) e reenvia quando a rede volta.
- Correção de tempo: `session_time_corrections` + `time_corrected_at` ; o filtro de métricas (`metrics.js`) exclui sessões corrigidas, anuladas e legadas.
- Treino esquecido: no app, ≥ 60 min sem atividade abre a recuperação; na API, a varredura marca e lembra a cada 60 min.

8.2 Ficha nova substitui a anterior e um treino ativo (R10 + R13)

`pro.js` : ao criar/publicar ficha para um aluno, as fichas ativas anteriores do mesmo aluno recebem `status=arquivado` , `superseded_by=<nova>` e `archived_at` , e em seguida `enforceOneActive(aluno)` pausa o programa do sistema ativo (`user_programs` → `switched`), sem apagar. `GET /api/care` devolve `plans` (ativa) e `plan_history` (somente leitura). Sessões em ficha antiga ou em algo que não é o treino ativo são recusadas (`ONE_ACTIVE`).

8.3 Decisão de acesso no servidor

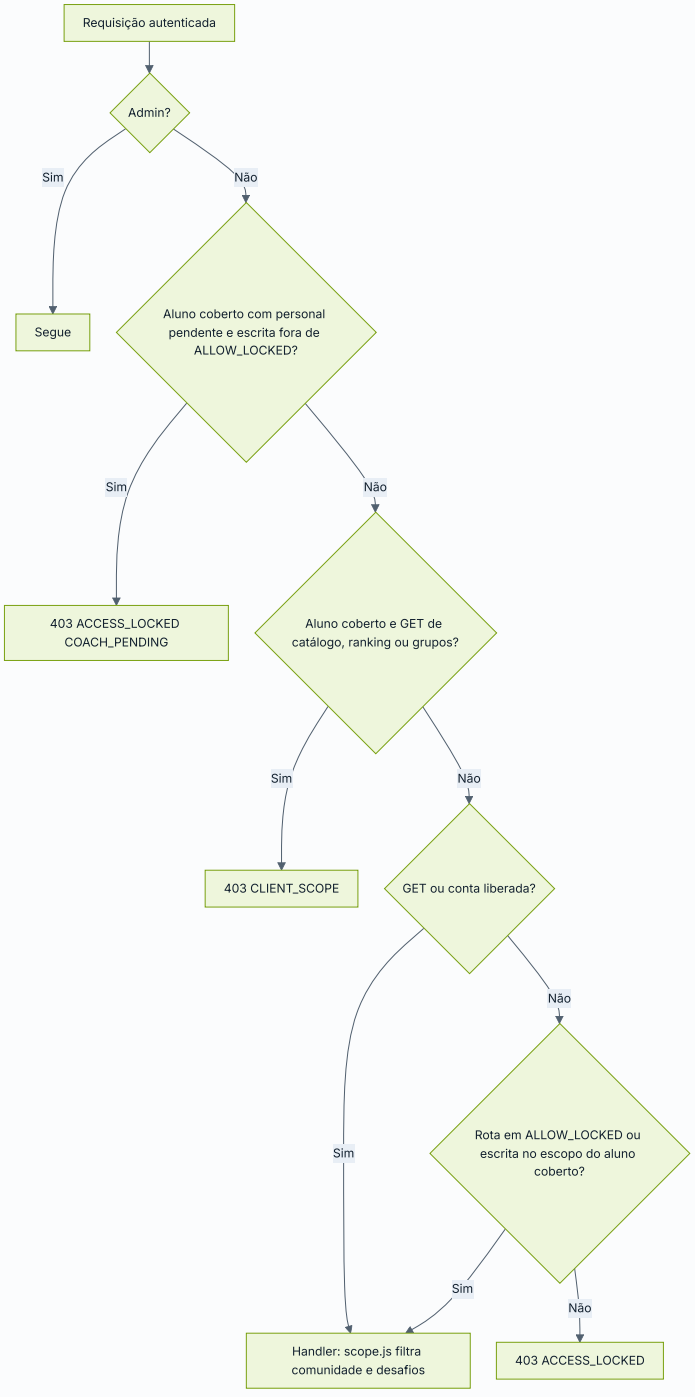


Figura 4 — accessGate + scope.js (R1–R9).

8.4 LGPD — exportação por e-mail

`POST /api/me/export-request`: confere senha (bcrypt) e e-mail da conta; limite 1/24 h (429 `RATE_LIMIT`); monta o JSON com `buildExport` (sem segredos); enfileira em `email_outbox` com cópia para `LGPD_COPY_EMAIL`; grava `data_export_requests` (IP truncado) e auditoria. Resposta 202 com `mail_configured`.

9 Service worker e atualização

Peça	Comportamento
Build	`vite.config.js` gera `BUILD_ID` (aaaammddhhmmss), injeta `__BUILD_ID__` no bundle, grava `dist/version.json` e troca o marcador no `sw.js`. Produção (15/10/2026): build `20261015233640`, cache `avagym-rt-v41-20261015233640`.
sw.js	`install → skipWaiting`; `activate` apaga caches antigos e faz `clients.claim`; `/api/*` nunca é cacheado; mídia `/img` e `/gif` cache-first; navegação e `version.json` sempre da rede (`no-store`); fallback offline para o cache/ `index.html` ; trata `push` e `notificationclick`.
pwa-update.js	Registro com `updateViaCache:"none"`; checa a cada 20 min, ao voltar ao foco e ao reconectar; compara `version.json` ; no `controllerchange` recarrega só fora do treino (`pwa-rules.inWorkout` : rota de treino/conclusão ou sessão local aberta) — senão mostra a barra “Nova versão disponível — atualiza ao terminar o treino.”; proteção contra laço via `sessionStorage`.
Deploy	Manter os assets do build anterior em `dist/assets` para quem está com a versão antiga aberta (evita 404 de chunk no meio do treino).

10 Segurança (rev. 2)

Entrou na main nos commits `3935401` e `63454ec` e está em produção. O comentário do código fala em "auditoria 2026-10", mas **não existe relatório de auditoria no repositório**; aqui está documentado só o que o código faz.

Controle	Detalhe
Cabeçalhos HTTP (API)	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Cross-Origin-Resource-Policy: same-origin` e, quando `ORIGIN` é https, `Strict-Transport-Security` (1 ano, subdomínios).
Limite de tentativas	Janela fixa em memória por IP (`cf-connecting-ip` → `x-forwarded-for` → socket): login 20 / 15 min; cadastro 10 / h; esqueci a senha 5 / 15 min; redefinir 10 / 15 min; trocar senha 10 / 15 min. Resposta 429 "Muitas tentativas..." com `Retry-After`. `RATE_LIMIT=off` desliga (só testes locais). Contadores zeram quando a API reinicia.
Exposição de rede	`HOST` configurável; produção usa `127.0.0.1`. Padrão `0.0.0.0` mantém o docker-compose.
nginx do docker	`server_tokens off` e os mesmos cabeçalhos (vale para o compose do repositório, não para a produção).
Senhas	bcrypt; senha provisória de 12 caracteres sorteados de um alfabeto de 56 símbolos, mostrada uma vez, sem expiração por tempo, troca obrigatória no 1º login (R8).

Lacuna observada: os cabeçalhos são da API. O HTML e os estáticos servidos pelo gateway `avagym-web` (fora do repositório) não recebem esses cabeçalhos. A CONFIRMAR levar os cabeçalhos ao gateway ou à Cloudflare

11 Testes

Suite	Onde	Resultado registrado
API V4.3 escopo (R1–R12)	`app/api/test/v43-escopo.mjs`	23 ok (rodado nesta revisão, banco local)
API V4.3 um treino (R13)	`app/api/test/v43-um-treino.mjs`	21 ok (rodado nesta revisão)
API V4.2	`app/api/test/v42.mjs`	34 ok (rodado nesta revisão; exige o personal de teste verificado e liberado no banco local)
API V4.1	`test/v41.mjs`	17 ok
API V4 fase 2	`test/v4-fase2.mjs`	46 ok
API V3/V4 e smoke	`test/smoke*.mjs`, `v3-*.mjs`, `v4-fase*.mjs`	Falhas pré-existentes conhecidas em smoke e v3-fase2/3/6 (iguais à linha de base)
Unidade (node --test)	`challenge-rules`, `challenge-score`, `ranking`	—
Frontend (vitest)	`src/**/*.test.js` (18 arquivos)	243/243 (rodado nesta revisão)
E2E (Playwright)	`e2e/v4-fase1..4.spec.js`, `v42.spec.js`	21/21 e 5/5
E2E V4.3	`e2e/tours.spec.js`, `theme-toggle.spec.js`, `um-treino.spec.js`	Registrados como aprovados em `docs/v4/UM-TREINO-ATIVO.md`; não rodados nesta revisão
Capturas	`e2e/manual.spec.js` (MANUAL=1), `ux-v41.spec.js` (UX=1)	Geração de prints

E2E e testes de API rodam **só** contra ambiente local (API :8787, Vite :4449, Postgres local). Contas de teste: domínio `@avadesk.local` e `@e2e.local`; `E2E_AUTO_UNLOCK_DOMAIN` libera contas de teste só fora de produção. Remoção: `scripts/remover-usuarios.mjs --e2e`. Roteiro: `avagym/v4/scripts/rodar-testes-api.sh`.

12 Deploy, backup e rollback

12.1 Atualização (roteiro)

1. Janela de pouco uso; pedir que os alunos terminem treinos abertos.
2. Backup obrigatório (abaixo) e conferência de `schema_migrations`.
3. Commit e push na `main` do GitHub primeiro; na VPS, `git pull` em `/opt/avagym` (repositório privado: exige autenticação na VPS) — decisão do dono, nunca deploy automático.
4. `cd app/api && npm ci` ; `cd app/frontend && npm ci && npm run build` mantendo os assets antigos.
5. `node db/migrate.js --no-seed` (seed só se quiser atualizar o catálogo).
6. `pm2 restart avagym-api avagym-web --update-env && pm2 save` ; conferir `pm2 logs` .
7. Verificar: `/api/health` , `/version.json` com o build novo, nome do cache no `sw.js` , cabeçalhos no-store, fluxos principais com contas reais do dono (sem testes automáticos em produção).

12.2 Backup

- Banco: `pg_dump -Fc` e conferência com `pg_restore -l` ; cópia fora da VPS.
- Arquivos: `DATA_DIR` (db.json, segredo, VAPID), `uploads` , `private` .
- Código: commit anterior salvo em arquivo; tgz do código e do `dist` ; cópia do gateway e do `pm2` .
- Exemplo documentado: backup da V4.1 em pasta de backups do servidor (db.dump com 119 tabelas, codigo.tgz, dist.tgz, gateway, pm2.json).
- Backup específico do deploy V4.2: A CONFIRMAR não documentado.
- Local (Windows): `app/api/db/backup.ps1` .

12.3 Rollback

Tipo	Como	Efeito
Rápido (código)	Parar PM2 → checkout do commit anterior → `npm ci` + build → restart	Banco novo mantido; migrations são aditivas e o código antigo ignora colunas novas
Completo (banco)	`pg_restore --clean --if-exists` do dump pré-deploy + rollback de código	Perde tudo gravado após o deploy — último recurso
Estrutural	Aplicar `db/down/0NN_down.sql` em ordem inversa (ex.: 035 → 034)	Apaga tabelas/colunas novas e seus dados; fazer novo dump antes

13 Configuração e variáveis de ambiente

13.1 Configuração pendente

Item	Situação	Como ativar
SMTP	Vazio: e-mails ficam `pendente` em `email_outbox`	Instalar `nodemailer` em app/api, preencher `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM` e reiniciar com `--update-env`
LGPD_COPY_EMAIL	Vazio: sem cópia ao encarregado	Definir o e-mail do encarregado de dados
Pagamento	Sem gateway	Implementar `PAYMENT.forUnlock/forProfessional` e usar `unlock_requests.payment_ref`
Modelo de mensalidades	implementado (R1–R13; ver PRD). "Em dia" = liberação manual do admin	Quando houver pagamento: ligar o gateway ao `access_unlocked_at`
Exemplo do campo de código	O texto de exemplo do campo "Código do seu personal" no login usa um código real	Trocar por um exemplo fictício

13.2 Variáveis (somente nomes)

Grupo	Variáveis
Servidor	`PORT`, `HOST` (novo), `RATE_LIMIT` (novo; `off` desliga), `ORIGIN`, `DATA_DIR`, `NODE_ENV`, `RP_ID`, `RP_NAME`, `DEFAULT_TENANT`, `SESSION_DAYS`, `ADMIN_UIDS`
Banco	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`
Autenticação	`JWT_SECRET`, `JWT_TTL_HOURS`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`
Push	`VAPID_SUBJECT`
Sessões	`SESSION_SWEEP`, `SESSION_REMINDER_MIN`
E-mail / LGPD	`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`, `LGPD_COPY_EMAIL`
Testes/scripts	`E2E_AUTO_UNLOCK_DOMAIN`, `E2E_PASSWORD`, `API`, `BASE`, `GLB_PATH`
Frontend (build)	`BUILD_ID`, `VITE_API_BASE`, `VITE_IMG_BASE`, `VITE_GIF_BASE`, `VITE_MOBILE`, `VITE_DEMO`, `OPENGYM_MEDIA_DIR` (dev)

> `DEPLOY-V4.md` diz que `SESSION_SWEEP=0` desliga a varredura, mas o código só desliga com `off`. A CONFIRMAR corrigir doc ou código.

14 Itens a confirmar

Resolvidos nesta revisão: pasta da aplicação (`/opt/avagym`), nomes do PM2 (`avagym-api` , `avagym-web`), contêiner (`avagym-postgres` , postgres 16), domínio (`academ`) e modelo de mensalidades (implementado).

#	Item	Detalhe
1	Gateway	Código fora do repositório; versionar e aplicar os cabeçalhos de segurança ao HTML
2	Contêiner Postgres	Volume e rotina de backup automático
3	Túnel Cloudflare	Nome do túnel e regras; cache da borda
4	Persistência fora do Postgres	Push em db.json, ponte legada, presença e contadores do limite de tentativas em memória
5	SESSION_SWEEP	“0” (doc) × “off” (código)
6	SMTP e LGPD_COPY_EMAIL	Provedor não definido; em produção o app avisa que o e-mail está sendo configurado
7	Pagamento	Gateway e preços não definidos
8	Backups dos deploys V4.2, V4.3 e rev. 2	Não registrados no repositório
9	migrate.js em banco vazio	Falha na 028 (dependência do seed)
10	Relatório de auditoria de segurança	Não existe no repositório; só os commits de correção
11	Liberação de contas / painel do usuário	Tipo exibido do personal como “Usuário normal”; alunos cobertos em “Aguardando”; “Liberação: Travada” + “Não se aplica” no aluno coberto; contagem da Assinatura do personal diferente do Limite de alunos; “Último acesso: nunca” para quem já entrou; estilo do “Selecionar todas” no desktop
12	Exemplo do campo de código no login	Usa código real de usuário